

Database Design

Database

Design

- The reasons for normalizations in database design
- Functional Dependencies
- Decomposition
- Normal Forms (NF)
 - 👉 1NF
 - 👉 2NF
 - 👉 3NF
 - 👉 BCNF
 - 👉 Multivalued Dependencies and 4NF

■ Overall Database Design Process

5.1

Informal Design Guidelines for Relational Databases

Informal Design Guidelines for Relational Databases ■

What is relational database design?

The grouping of attributes to form "good" relation schemas

- Two levels of relation schemas
 - 👉 The logical "user view" level
 - 👉 The storage "base relation" level
- Design is concerned mainly with base relations
- What are the criteria for "good" base relations?

5.2

Pitfalls in Relational Database Design

Pitfalls in Relational Database Design

- Relational database design requires that we find a “good” collection of relation schemas. A bad design may lead to

- 👉 Repetition of Information.

- 👉 Inability to represent certain information.

- 👉 Possibility of inconsistency.

- Design Goals:

- 👉 Avoid redundant data

- 👉 Ensure that relationships among attributes are represented

- 👉 Facilitate the checking for violation of database integrity constraints.

Example

Example

- Consider the relation schema:

Lending-schema = (branch-name, branch-city, assets, customer-name, loan-number, amount)

<i>branch-name</i>	<i>branch-city</i>	<i>assets</i>	<i>customer-name</i>	<i>loan-number</i>	<i>amount</i>
Downtown	Brooklyn	9000000	Jones	L-17	1000
Redwood	Palo Alto	2100000	Smith	L-23	2000
Perryridge	Horseneck	1700000	Hayes	L-15	1500
Downtown	Brooklyn	9000000	Jackson	L-14	1500

- Redundancy:

📌 Data for *branch-name*, *branch-city*, *assets* are repeated for each loan that a branch makes.

- Waste space for storage
- Introduce **update anomalies**

5.4

Update Anomalies

Update Anomalies

■ Insertion anomalies

- 👉 To insert a new loan tuple, we must include the attribute values for the branch;
- 👉 It's difficult to insert a new branch which has no loan or customer yet. The only way to do that is to place null values in the attributes for loan or customer. When the first loan (associated with a customer) is assigned to that branch, we don't need the tuple with null values anymore.

■ Deletion anomalies:

👉 If we delete a tuple which happens to represent the last loan for a particular branch, the information concerning that branch is lost from the database.

■ Modification anomalies:

👉 If we change the value of one of the attributes of a particular branch, we must update all tuples for that branch; otherwise, the database will become inconsistent.

5.5

Decomposition

Decomposition

- Decompose the relation schema *Lending-schema* into:

Branch-schema = (branch-name, branch-city, assets)

*Loan-info-schema = (customer-name, loan-number,
branch-name, amount)*

- All attributes of an original schema (R) must appear in the decomposition (R_1, R_2):

$$R = R_1 \cup R_2$$

- Lossless-join decomposition.

For all possible relations r on schema R

$$r = \prod_{R_1} (r) \prod_{R_2} (r)$$

Guideline to Database Schema Design

Guideline to

Database Schema Design



GUIDELINE 1: Informally, each tuple in a relation should represent one entity or relationship instance.

Bottom Line: Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.

GUIDELINE 2: Design a schema that does not suffer from the insertion, deletion and update anomalies.

GUIDELINE 3: Relations should be designed such that their tuples will have as few NULL values as possible

GUIDELINE 4: The relations should be designed to satisfy the lossless join condition. No spurious tuples should be generated by doing a natural-join of any relations.

5.7



**Goal — Devise a Theory for the
Following**



- Decide whether a particular relation R is in “good” form.

■ In the case that a relation R is not in “good” form, decompose it into a set of relations $\{R_1, R_2, \dots, R_n\}$ such that

👉 each relation is in good form

👉 the decomposition is a lossless-join

decomposition ■ Our theory is based on:

👉 functional dependencies

👉 multivalued dependencies



Functional Dependencies (FD)



Constraints on the set of legal relations.

- Require that the value for a certain set of attributes determines uniquely the value for another set of attributes.
- A functional dependency is a generalization of the notion of a *key*.

50



Functional Dependencies (FD)

A	B	C	D
A1	B1	C1	D1

A1	B2	C1	D2
A2	B2	C2	D2
A2	B2	C2	D3
A3	B3	C2	D4

5.10

roll_no	name	dept_name	dept_building
42	abc	CO	A4
43	pqr	IT	A3
44	xyz	CO	A4
45	xyz	IT	A3
46	mno	EC	B2

47	jkl	ME	B2
----	-----	----	----

From the above table we can conclude some valid functional dependencies:
 $\text{roll_no} \rightarrow \{ \text{name}, \text{dept_name}, \text{dept_building} \}$, $\rightarrow$ Here, roll_no can determine values of fields name, dept_name and dept_building, hence a valid Functional dependency

$\text{roll_no} \rightarrow \text{dept_name}$, Since, roll_no can determine whole set of {name, dept_name, dept_building}, it can determine its subset dept_name also.

$\text{dept_name} \rightarrow \text{dept_building}$, Dept_name can identify the dept_building accurately, sincdepartments with different dept_name will also have a different dept_building

More valid functional dependencies: $\text{roll_no} \rightarrow \text{name}$, $\{\text{roll_no}, \text{name}\} \twoheadrightarrow \{\text{dept_name},$

5.11

Here are some invalid functional dependencies:

$\text{name} \rightarrow \text{dept_name}$ Students with the same name can have different dept_name, hence this is not a valid functional dependency.

$\text{dept_building} \rightarrow \text{dept_name}$ There can be multiple departments in the same building, For example, in the above table departments ME and EC are in the same building B2, hence $\text{dept_building} \rightarrow \text{dept_name}$ is an invalid functional dependency.

More invalid functional dependencies: $\text{name} \rightarrow \text{roll_no}$, $\{\text{name}, \text{dept_name}\} \rightarrow \text{roll_no}$, $\text{dept_building} \rightarrow \text{roll_no}$, etc.

5.12

Armstrong's axioms/properties of functional dependencies:

Reflexivity: If Y is a subset of X , then $X \rightarrow Y$ holds by reflexivity rule For example, $\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$ is valid.

Augmentation: If $X \rightarrow Y$ is a valid dependency, then $XZ \rightarrow YZ$ is also valid by the augmentation rule.

For example, If $\{\text{roll_no}, \text{name}\} \rightarrow \text{dept_building}$ is valid, hence $\{\text{roll_no}, \text{name}, \text{dept_name}\} \rightarrow \{\text{dept_building}, \text{dept_name}\}$ is also valid.→

Transitivity: If $X \rightarrow Y$ and $Y \rightarrow Z$ are both valid dependencies, then $X \rightarrow Z$ is also valid by the Transitivity rule.

For example, $\text{roll_no} \rightarrow \text{dept_name}$ & $\text{dept_name} \rightarrow \text{dept_building}$, then $\text{roll_no} \rightarrow \text{dept_building}$ is also valid

5.13

Types of Functional dependencies in DBMS:

1. Trivial functional dependency
2. Non-Trivial functional dependency
3. Multivalued functional dependency

4. Transitive functional dependency

1. Trivial Functional Dependency

In Trivial Functional Dependency, a dependent is always a subset of the determinant.

i.e. If $X \rightarrow Y$ and Y is the subset of X , then it is called trivial functional dependency. For example,

roll_no name age

42 abc 17

43 pqr 18

44 xyz 18

Here, $\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent **name** is a subset of determinant set $\{\text{roll_no}, \text{name}\}$. Similarly, $\text{roll_no} \rightarrow \text{roll_no}$ is also an example of trivial functional dependency.

5.14

• =

In **Non-trivial functional dependency**, the dependent is strictly not a

subset of the determinant.

i.e. If $X \rightarrow Y$ and **Y is not a subset of X**, then it is called Non-trivial functional dependency.

For example,

Here, **roll_no** $\rightarrow$ **name** is a non-trivial functional dependency, since the dependent **name** is **not a subset of** determinant **roll_no**

Similarly, **{roll_no, name}** $\rightarrow$ **age** is also a non-trivial functional dependency, since **age** is **not a subset of** {roll_no, name}

roll_no name age

42 abc 17

43 pqr 18

44 xyz 18

5.15

3. Multivalued Functional Dependency

In **Multivalued functional dependency**, entities of the dependent set are **not dependent on each other**.

i.e. If $a \rightarrow \{b, c\}$ and there exists **no functional dependency** between **b** and **c**,

then it is called a **multivalued functional dependency**.

For example,

Here, $\text{roll_no} \rightarrow \{\text{name}, \text{age}\}$ is a multivalued functional dependency, since the

dependents **name** & **age** are **not dependent** on each other (i.e. $\text{name} \rightarrow \text{age}$ or $\text{age} \rightarrow \text{name}$ doesn't exist !)

roll_no name age

42 abc 17

43 pqr 18

44 xyz 18

45 abc 19

5.16

4. Transitive Functional Dependency

In transitive functional dependency, dependent is indirectly dependent on determinant. i.e. If $\mathbf{a} \rightarrow \mathbf{b}$ & $\mathbf{b} \rightarrow \mathbf{c}$, then according to axiom of transitivity, $\mathbf{a} \rightarrow \mathbf{c}$. This is a **transitive functional dependency**

For example,

Here, $\mathbf{enrol_no} \rightarrow \mathbf{dept}$ and $\mathbf{dept} \rightarrow \mathbf{building_no}$,

Hence, according to the axiom of transitivity, $\mathbf{enrol_no} \rightarrow \mathbf{building_no}$ is a valid functional dependency. This is an indirect functional dependency, hence called

Transitive functional dependency.

enrol_no name dept building_no 42 abc CO 4

43 pqr EC 2

44 xyz IT 1

45 abc EC 2



Functional Dependencies (Cont.)

- Let R be a relation schema

$$\alpha \subseteq R \text{ and } \beta \subseteq R$$

■ The functional dependency

$$\alpha \rightarrow \beta$$

holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α , they also agree on the attributes β . That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

■ Example: Consider $r(A,B)$ with the following instance of r .

1	4
1	5
3	7

■ On this instance, $A \rightarrow B$ does **NOT** hold, but $B \rightarrow A$ does hold.



Examples of FD

constraints



- social security number determines employee name
SSN $\rightarrow$ ENAME
- project number determines project name and location
PNUMBER $\rightarrow$ {PNAME, PLOCATION}
- employee ssn and project number determines the hours per week that the employee works on

the project

$\{\text{SSN}, \text{PNUMBER}\} \rightarrow \text{HOURS}$

How about $\{\text{Teacher}, \text{Course}, \text{Text}\}$?

5.19



Functional Dependencies (Cont.)



K is a superkey for relation schema R if and only if $K \rightarrow R$

■ K is a candidate key for R if and only if

👉 $K \rightarrow R$, and

👉 for no $\alpha \subset K$, $\alpha \rightarrow R$

- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:

*Loan-info-schema = (customer-name,
loan-number, branch-name,
amount).*

We expect this set of functional dependencies to hold:

loan-number $\rightarrow$ *amount*
loan-number $\rightarrow$ *branch-name*

but would not expect the following to hold:

loan-number $\rightarrow$ *customer-name*

5.20





Use of Functional Dependencies ■

We use functional dependencies to:

👉 test relations to see if they are legal under a given set of functional dependencies.

□ If a relation r is legal under a set F of functional dependencies, we say that r **satisfies** F .

👉 specify constraints on the set of legal relations

□ We say that F **holds on** a relation schema R if all legal relations on R satisfy the set of functional dependencies F .

■ Note: A specific instance of a relation schema may satisfy a functional dependency even if the functional dependency does not hold on all legal instances. For example, a specific instance of *Loan-schema* may, by chance, satisfy $\text{loan-number} \rightarrow \text{customer-name}$.

5.21



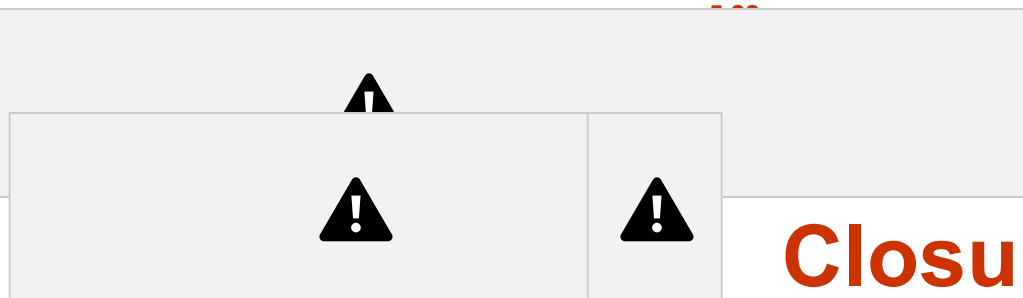
Functional Dependencies (Cont.) ■

A functional dependency is **trivial** if it is satisfied by all instances of a relation

👉 *E.g.*

- *customer-name, loan-number → customer name*
- *customer-name → customer-name*

👉 In general, $\alpha \rightarrow \beta$ is trivial if and only if $\beta \subseteq \alpha$



Closure of a Set of Functional Dependencies

- Given a set F set of functional dependencies, there are certain

other functional dependencies that are logically implied by F . 📌

E.g. If $A \rightarrow B$ and $B \rightarrow C$, then we can infer that $A \rightarrow C$

■ The set of all functional dependencies logically implied by F is the *closure* of F .

■ We denote the *closure* of F by F^+ .

■ We can find all of F^+ by applying Armstrong's Axioms:

📌 if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ (**reflexivity**)

📌 if $\alpha \rightarrow \beta$, then $\gamma \alpha \rightarrow \gamma \beta$ (**augmentation**)

📌 if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$ (**transitivity**)

■ These rules are

📌 **sound** (generate only functional dependencies that actually hold) and

📌 **complete** (generate all functional dependencies that hold).



Example

■ $R = (A, B, C, G, H, I)$

$F = \{ A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H \}$

■ some members of F^+

👉 $A \rightarrow H$

□ by transitivity from $A \rightarrow B$ and $B \rightarrow H$

👉 $AG \rightarrow I$

□ by augmenting $A \rightarrow C$ with G , to get $AG \rightarrow CG$
and then transitivity with $CG \rightarrow I$

👉 $CG \rightarrow HI$

□ from $CG \rightarrow H$ and $CG \rightarrow I$: “union rule” can be inferred from
— definition of functional dependencies, or

- Augmentation of $CG \rightarrow I$ to infer $CG \rightarrow CGI$, augmentation of $CG \rightarrow H$ to infer $CGI \rightarrow HI$, and then transitivity

5.24



+



To

Procedure for Computing F^+

compute the closure of a set of functional dependencies F :

$$F^+ = F$$

repeat

for each functional dependency f in F^+

apply reflexivity and augmentation rules on f

add the resulting functional dependencies to F^+ **for**

each pair of functional dependencies f_1 and f_2 in F^+ **if**

f_1 and f_2 can be combined using transitivity
 then add the resulting functional dependency to
 F^+ **until** F^+ does not change any further

NOTE: We will see an alternative procedure for this task later



Closure of Functional Dependencies (Cont.)

- We can further simplify manual computation of F^+ by using

the following additional rules.

☞ If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta \vee \gamma$ holds (**union**)

☞ If $\alpha \rightarrow \beta \vee \gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds
(**decomposition**)

☞ If $\alpha \rightarrow \beta$ holds and $\gamma \wedge \beta \rightarrow \delta$ holds, then $\alpha \wedge \gamma \rightarrow \delta$ holds (**pseudo transitivity**)

☞ If $\alpha \rightarrow \beta$ holds and $\gamma \rightarrow \delta$ holds, then $\alpha \wedge \gamma \rightarrow \beta \delta$ holds
(**composition**)

☞ If $\alpha \rightarrow \beta$ holds and $\gamma \rightarrow \delta$ holds, then $\alpha \cup (\gamma - \beta) \rightarrow \beta \delta$ holds
(**General unification**)

The above rules can be inferred from Armstrong's axioms.



Closure of



Attribute Sets

- Given a set of attributes α , define the *closure* of α under F (denoted by α^+) as the set of all attributes that are functionally determined by α under F :

$$\alpha \rightarrow \beta \text{ is in } F^+ \Leftrightarrow \beta \subseteq \alpha^+$$

- Algorithm to compute α^+ , the closure of α under F

result := α ;

while (changes to *result*) **do**

for each $\beta \rightarrow \gamma$ **in** F **do**

begin

if $\beta \subseteq \text{result}$ **then** *result* := *result* $\cup \gamma$

end



Example of Attribute Set Closure

■ $R = (A, B, C, G, H, I)$

■ $F = \{A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H\}$

■ $(AG)^+$

1. *result* = AG

2. $result = ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
3. $result = ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
4. $result = ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)

■ Is AG a candidate key?

1. Is AG a super key?
 1. Does $AG \rightarrow R$?
2. Is any subset of AG a superkey?
 1. Does $A^+ \rightarrow R$?
 2. Does $G^+ \rightarrow R$?

5.28



Uses of

Attribute Closure



There are several uses of the attribute closure

algorithm: ■ Testing for superkey:

👉 To test if α is a superkey, we compute α^+ , and check if α^+ contains all attributes of R .

■ Testing functional dependencies

👉 To check if a functional dependency $\alpha \rightarrow \beta$ holds (or, in other words, is in F^+), just check if $\beta \subseteq \alpha^+$.

👉 That is, we compute α^+ by using attribute closure, and then check if it contains β .

👉 Is a simple and cheap test, and very useful

■ Computing closure of F

👉 For each $\gamma \subseteq R$, we find the closure γ^+ , and for each $S \subseteq \gamma^+$, we output a functional dependency $\gamma \rightarrow S$.



Irreducible (Minimal) Equivalent ■

Sets of functional dependencies may have redundant dependencies that can be inferred from the others

👉 Eg: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$ 👉

Parts of a functional dependency may be redundant

□ E.g. on RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to

$\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$

□ E.g. on LHS: $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ can be simplified to

$\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$

■ Intuitively, an irreducible equivalent of F is a “minimal” set of

functional dependencies equivalent to F , with no redundant dependencies or having redundant parts of dependencies

5.30



Extraneous Attributes

- Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F .
 - 👉 Attribute A is **extraneous** in α if $A \in \alpha$
and $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$ logically implies F
 - 👉 Attribute A is **extraneous** in β if $A \in \beta$
and the set of functional dependencies
 $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F .
- *Note:* implication in the opposite direction is trivial in each of

the cases above, since a “stronger” functional dependency always implies a weaker one

■ Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$

👉 B is extraneous in $AB \rightarrow C$ because $A \rightarrow C$ logically implies $AB \rightarrow C$.

■ Example: Given $F = \{A \rightarrow C, AB \rightarrow CD\}$

👉 C is extraneous in $AB \rightarrow CD$ since $A \rightarrow C$ can be inferred even after deleting C

5.31



Testing Extraneous Attributes and FDs



■ Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F .

👉 To test if attribute $A \in \alpha$ is extraneous in α

1. compute $(\alpha - \{A\})^+$ using the dependencies in F
2. check that $(\alpha - \{A\})^+$ contains α ; if it does, A is extraneous 👉

To test if attribute $A \in \beta$ is extraneous in β

1. compute α^+ using only the dependencies in $F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$,
2. check that α^+ contains A ; if it does, A is extraneous

👉 To test if $\alpha \rightarrow \beta$ is redundant

1. compute α^+ using only the dependencies in $F' = (F - \{\alpha \rightarrow \beta\})$,
2. check that α^+ contains all attributes in β ; if it does, $\alpha \rightarrow \beta$ is redundant

5.32



Irreducible

equivalent



- A FD set F is called a **cover** of a FD set G if F^+ is a superset of G^+ .
- Two FD sets F and G are called **equivalent** if $F^+ = G^+$.
- A FD set F is **irreducible (minimal)** if:

- 👉 The right side (the dependent) of each FD in F has just one attribute, and
- 👉 The left side (the determinant) of each FD in F is irreducible, i.e., no attribute can be discarded from the determinant without changing the

closure F^+ , and

👉 No FD in F can be discarded from F without changing the closure F^+ .

■ A FD set F is called an **irreducible equivalent** (or canonical/minimal cover) of a FD set G if $F^+ = G^+$ and F is irreducible.

👉 Each FD set F has an irreducible equivalent

👉 The irreducible equivalents of F may not be unique

5.33



Irreducible



equivalent

■ To compute an irreducible equivalent for F : Rewrite

the FDs such that each has a singleton right side;

repeat

Find a FD $\alpha \rightarrow \beta$ with an extraneous attribute in α ; If an extraneous attribute is found, delete it from the left side of $\alpha \rightarrow \beta$;

Find a FD which is redundant and delete it from F ; **until** F does not change



Example



- $R = (A, B, C)$
 $F = \{A \rightarrow BC,$
 $B \rightarrow C,$
 $A \rightarrow B,$
 $AB \rightarrow C\}$
- Decompose $A \rightarrow BC$.
 - 👉 We get $\{A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$
 - 👉 Remove the duplicate $A \rightarrow B$, we get $\{A \rightarrow B, A \rightarrow C, B \rightarrow C, AB \rightarrow C\}$
- $A \rightarrow C$ is redundant; remove it.
 - 👉 Set is now $\{A \rightarrow B, B \rightarrow C, AB \rightarrow C\}$
- $AB \rightarrow C$ is redundant; remove it.
 - 👉 Set is now $\{A \rightarrow B, B \rightarrow C\}$
- The irreducible equivalent of F is:
$$\{A \rightarrow B,$$

$$B \rightarrow C\}$$

5.35



Goals of

Normalization



- Decide whether a particular relation R is in “good” form.
- In the case that a relation R is not in “good” form, decompose it into a set of relations $\{R_1, R_2, \dots, R_n\}$ such that
 - 👉 each relation is in good form

👉 the decomposition is a lossless-join decomposition ■ Our theory is based on:

👉 functional dependencies

👉 multivalued dependencies

5.36



Decomposition



- Decompose the relation schema *Lending-schema* into:
Branch-schema = (*branch-name*, *branch-city*, *assets*)

Loan-info-schema = (*customer-name*, *loan-number*,
branch-name, *amount*) ■ All attributes of an original
schema (R) must appear in the decomposition (R_1, R_2):

$$R = R_1 \cup R_2$$

- Lossless-join decomposition.

For all possible relations r on schema R

$$r = \prod_{R_1}(r) \prod_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is lossless join if and only if at least one of the following dependencies is in F^+ (F is the set of FDs satisfied by R):

👉 $R_1 \cap R_2 \rightarrow R_1$

👉 $R_1 \cap R_2 \rightarrow R_2$



Example of Lossy-Join



Decomposition

- Lossy-join decompositions result in information loss.
- Example: Decomposition of $R = (A, B)$

$$R_2 = (A) \quad R_2 = (B)$$

$A \ B$	A	B
$\alpha \ \alpha \ \beta$	1 2 1	$\alpha \ \beta$ 1 2

$$\Pi_{A(r)} \Pi_{B(r)} \quad \Pi_{A(r)} \Pi_{B(r)}$$

$\alpha \ \alpha$	1 2
$\beta \ \beta$	1 2



Normalization Using Functional Dependencies



■ When we decompose a relation schema R with a set of functional dependencies F into $R_1, R_2, \dots, R_n$ we want

👉 **Lossless-join decomposition**: Otherwise decomposition would result in information loss.

👉 **No redundancy**: The relations R_i preferably should be in either Boyce Codd Normal Form or Third Normal Form.

👉 **Dependency preservation**: Let F_i be the set of dependencies F^+ that include only attributes in R_i .

□ Preferably the decomposition should be **dependency preserving**,

that is, $(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$

- Otherwise, checking updates for violation of functional dependencies may require computing joins, which is expensive.

5.39



Testing for Dependency Preservation

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $R_1, R_2, \dots, R_n$ we apply the following simplified test (with attribute closure done w.r.t. F)

👉 *result* = α

while (changes to *result*) do

for each R_i in the decomposition

$t = (result \cap R_i)^+ \cap R_i$

result = *result* $\cup$ t

👉 If *result* contains all attributes in β , then the functional dependency $\alpha \rightarrow \beta$ is preserved.

- We apply the test on all dependencies in F to check if a decomposition is dependency preserving
- This procedure takes polynomial time, instead of the exponential time required to compute F^+ and $(F_1 \cup F_2 \cup \dots \cup F_n)^+$



First

Normal Form (1NF)



- Domain is **atomic** if its elements are considered to be indivisible units
 - 👉 Examples of non-atomic domains:
 - Composite attributes
 - Multi-valued attributes
- A relational schema R is in **first normal form** if the domains of all attributes of R are atomic
- Non-atomic values complicate system design and

data storage

5.42
5.43





Second Normal Form



Definitions:

- **Prime attribute** - attribute that is member of the primary key K
- **Full functional dependency** - a FD $Y \rightarrow Z$ where removal of any attribute from Y means the FD does not hold any more

Examples: - $\{SSN, PNUMBER\} \rightarrow HOURS$ is a full FD since neither $SSN \rightarrow HOURS$ nor $PNUMBER \rightarrow HOURS$ hold

- $\{SSN, PNUMBER\} \rightarrow ENAME$ is *not* a full FD (it is called a *partial dependency*) since $SSN \rightarrow ENAME$ also holds

5.45



Second

Normal Form



- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on the primary key
- R can be decomposed into 2NF relations via the process

of 2NF normalization

■ E.g.

👉 $R = (A, B, C, D)$

$F = \{AB \rightarrow CD, A \rightarrow D\}$

👉 Decomposition: $R_1 = (A, D), R_2 = (A, B, C)$

👉 Lossless-join decomposition:

$R_1 \cap R_2 = \{A\}$ and $A \rightarrow D$

👉 Dependency preserving





Third Normal Form

Definition:

- **Transitive functional dependency** - a FD $X \rightarrow Z$ that can be derived from two FDs $X \rightarrow Y$ and $Y \rightarrow Z$

Examples:

- $SSN \rightarrow DMGRSSN$ is a *transitive* FD since $SSN \rightarrow DNUMBER$ and $DNUMBER \rightarrow DMGRSSN$ hold
- $SSN \rightarrow ENAME$ is *non-transitive* since there is no set of attributes X where $SSN \rightarrow X$ and $X \rightarrow$

ENAME

5.48



Third Normal Form



- A relation schema R is in **third normal form (3NF)** if it is in 2NF *and* no non-prime attribute A in R is transitively dependent on the primary key
- R can be decomposed into 3NF relations via the

process of 3NF normalization

NOTE:

In $X \rightarrow Y$ and $Y \rightarrow Z$, with X as the primary key, we consider this a problem only if Y is not a candidate key. When Y is a candidate key, there is no problem with the transitive dependency .

E.g., Consider EMP (SSN, Emp#, Salary).

Here, $SSN \rightarrow Emp\# \rightarrow Salary$ and Emp# is a candidate key.

5.49



General Normal Form Definitions (For



Multiple Keys)

- The above definitions consider the primary key only
- The following more general definitions take into account relations with multiple candidate keys
- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on *every key* of R

5.50



General Normal Form Definitions

A relation schema R is in **third normal form (3NF)** if whenever a FD $X \rightarrow A$ holds in R , then either:

(a) X is a superkey of R , or

(b) A is a prime attribute of R

(If X is not a superkey,

(1) X is a proper subset of some key K , then a partial FD

(2) X is not a proper subset of any key, then $K \rightarrow X \rightarrow A$, a transitive FD)





3NF (Cont.)



■ Example



$R = (J, K, L)$

$F = \{JK \rightarrow L, L \rightarrow K\}$



Two candidate keys: JK and JL



R is in 3NF

$JK \rightarrow L$ JK is a superkey

$L \rightarrow K$ K is contained in a candidate key

5.53



Testing for 3NF



- Optimization: Need to check only FDs in F , need not check all FDs in F^+ .
- Use attribute closure to check, for each dependency α

→ β , if α is a superkey.

- If α is not a superkey, we have to verify if each attribute in β is contained in a candidate key of R

👉 this test is rather more expensive, since it involve finding candidate keys

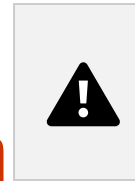
👉 testing for 3NF has been shown to be NP-hard

👉 Interestingly, decomposition into third normal form (described shortly) can be done in polynomial time

5.54



3NF Decomposition Algorithm



Let F_c

be a canonical (or minimal) cover for F ;

$i := 0$;

for each functional dependency $\alpha \rightarrow \beta$ in F_c **do**

if none of the schemas R_j , $1 \leq j \leq i$ contains $\alpha \beta$

then begin

$i := i + 1$;

$R_i := \alpha \beta$

end

if there any remaining attributes that have not been placed in any relations

then begin

$i := i + 1$;

$R_i :=$ remaining attributes in R ;

end

return $(R_1, R_2, \dots, R_i)$

Attribute preservation.

5.55



Example



$R = ABCD, A \rightarrow B,$

$C \rightarrow B, \text{Key: } ACD$

Decompositions: AB, BC,

D Lossy decomposition



3NF

Decomposition Algorithm

Let F_c be a canonical cover for F ;
 $i := 0$;

```

for each functional dependency  $\alpha \rightarrow \beta$  in  $F_c$  do
  if none of the schemas  $R_j$ ,  $1 \leq j \leq i$  contains  $\alpha \beta$ 
    then begin
       $i := i + 1$ ;
       $R_i := \alpha \beta$ 
    end
  if none of the schemas  $R_j$ ,  $1 \leq j \leq i$  contains a candidate key for  $R$ 
    then begin
       $i := i + 1$ ;
       $R_i :=$  any candidate key for  $R$ ;
    end
return  $(R_1, R_2, \dots, R_i)$ 

```



Example



$R = ABCD, A \rightarrow B,$

$C \rightarrow B, \text{Key: } ACD$

Decompositions: AB, BC, ACD

5.58





3NF Decomposition Algorithm (Cont.)

■ Above algorithm ensures:

- 👉 each relation schema R_i is in 3NF
- 👉 decomposition is dependency preserving and lossless-join

5.59



Example



- Relation schema:

Banker-info-schema = (branch-name, customer

name, banker-name, office-number)

- The functional dependencies for this relation schema are:

banker-name $\rightarrow$ *branch-name* *office-number*

customer-name *branch-name* $\rightarrow$ *banker-name*

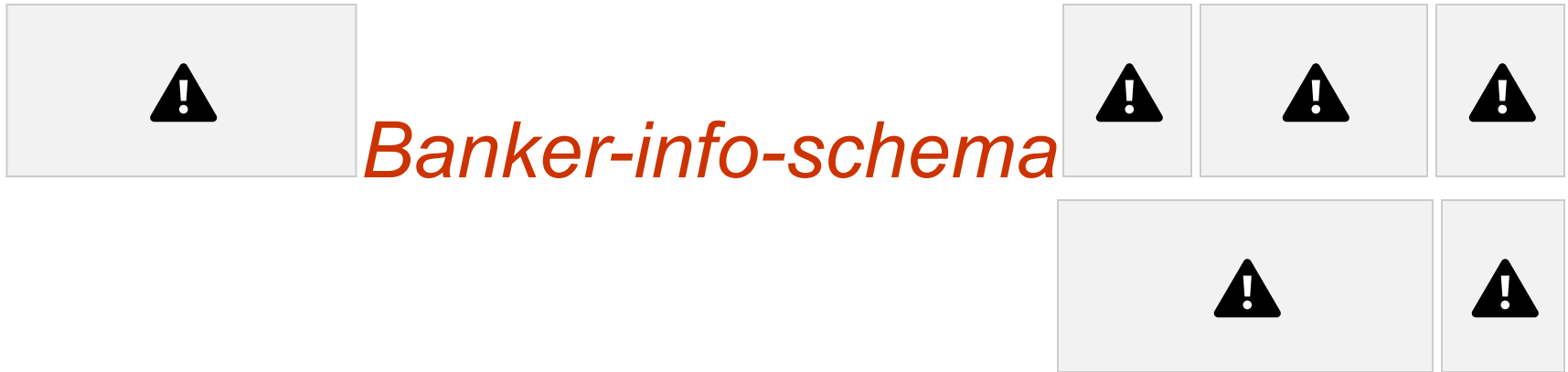
- The key is:

{customer-name, branch-name}

5.60



Applying 3NF to



- The **for** loop in the algorithm causes us to include the following schemas in our decomposition:

Banker-office-schema = (banker-name, branch-name, office-number)

Banker-schema = (customer-name, branch name, banker-name)

- Since *Banker-schema* contains a candidate key for *Banker-info-schema*, we are done with the

decomposition process.

5.61



3NF (Cont.)



■ Example

👉 $R = (J, K, L)$

$F = \{JK \rightarrow L, L \rightarrow K\}$

👉 Two candidate keys: JK and JL

👉 R is in 3NF

$JK \rightarrow L$ JK is a superkey

$L \rightarrow K$ K is contained in a candidate key $J L$

K

j_1	j_2	l_1	l_1	k_1	k_1
j_3	$?$	l_1	l_2	k_2	
			k_1		

5.62



BCNF (Boyce-Codd



Normal Form)



■ A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an FD $X \rightarrow A$ holds in R , then X is a superkey of R

■ Each normal form is strictly stronger than the previous one

👉 Every 2NF relation is in 1NF

👉 Every 3NF relation is in 2NF

👉 Every BCNF relation is in 3NF

■ There exist relations that are in 3NF but not in BCNF

■ The goal is to have each relation in BCNF (or 3NF)

Note: BCNF is most desirable form, no redundancy

detected by FDs.

5.63

Example

- $R = (A, B, C)$
 $F = \{A \rightarrow B$
 $B \rightarrow C\}$
Key = $\{A\}$
- R is not in BCNF
- Decomposition $R_1 = (A, B), R_2 = (B, C)$
 - 👉 R_1 and R_2 in BCNF
 - 👉 Lossless-join decomposition
 - 👉 Dependency preserving

5.64



5.65





- ## Achieving the BCNF by Decomposition
- Two FDs exist in the relation TEACH:
 - fd1: { student, course} $\rightarrow$ instructor
 - fd2: instructor $\rightarrow$ course
 - {student, course} is a candidate key for this relation and that the dependencies shown follow the pattern in the previous slide. So this relation is in 3NF but not in BCNF

- A relation **NOT** in BCNF should be decomposed so as to meet this property, while possibly forgoing the preservation of all functional dependencies in the decomposed relations.

5.67



Achieving the BCNF by Decomposition ■

Three possible decompositions for relation TEACH 1. {student, instructor} and {student, course}

2. {course, instructor} and {course, student}

3. {instructor, course } and {instructor, student}

- All three decompositions will lose fd1. We have to settle for sacrificing the functional dependency preservation. But we cannot sacrifice the non-additivity property after decomposition.
- Out of the above three, only the 3rd decomposition will not generate spurious tuples after join.(and hence has the non additivity property).



5.68

Testing for BCNF

- To check if a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF

1. compute α^+ (the attribute closure of α), and
2. verify that it includes all attributes of R , that is, it is a superkey of R .

■ **Simplified test:** To check if a relation schema R with a given set of functional dependencies F is in BCNF, it suffices to check only the dependencies in the given set F for violation of BCNF, rather than checking all dependencies in F^+ .

👉 We can show that if none of the dependencies in F causes a violation of BCNF, then none of the dependencies in F^+ will cause a violation of BCNF either.

■ However, using only F is **incorrect** when testing a relation in a decomposition of R

👉 E.g. Consider $R(A, B, C, D)$, with $F = \{A \rightarrow B, B \rightarrow C\}$

- Decompose R into $R_1(A, B)$ and $R_2(A, C, D)$
- Neither of the dependencies in F contain only attributes from (A, C, D) so we might be misled into thinking R_2 satisfies BCNF.
- In fact, dependency $A \rightarrow C$ in F^+ shows R_2 is not in BCNF.



BCNF Decomposition Algorithm

```
result := {R};  
done := false;  
compute  $F^+$ ;  
while (not done) do  
    if (there is a schema  $R_i$  in result that is not in BCNF)  
        then begin  
            let  $\alpha \rightarrow \beta$  be a nontrivial functional  
                dependency that holds on  $R_i$   
                such that  $\alpha \rightarrow R_i$  is not in  $F^+$ ,  
                and  $\alpha \cap \beta = \emptyset$ ;  
            result := (result -  $R_i$ )  $\cup$  ( $R_i - \beta$ )  $\cup$  ( $\alpha, \beta$ );  
        end
```

else *done* := **true**;

Note: each R_i is in BCNF, and decomposition is lossless-join.



Example of BCNF Decomposition

- $R = (\text{branch-name}, \text{branch-city}, \text{assets}, \text{customer-name}, \text{loan-number}, \text{amount})$
 $F = \{\text{branch-name} \rightarrow \text{assets} \text{ branch-city}$
 $\text{loan-number} \rightarrow \text{amount} \text{ branch-name}\}$
Key = $\{\text{loan-number}, \text{customer-name}\}$

■ Decomposition

👉 $R_1 = (\text{branch-name}, \text{branch-city}, \text{assets})$

👉 $R_2 = (\text{branch-name}, \text{customer-name}, \text{loan-number}, \text{amount})$

👉 $R_3 = (\text{branch-name}, \text{loan-number}, \text{amount})$

👉 $R_4 = (\text{customer-name}, \text{loan-number})$

■ Final decomposition

R_1, R_3, R_4



Testing Decomposition for BCNF

■ To check if a relation R_i in a decomposition of R is in BCNF, 👉

Either test R_i for BCNF with respect to the **restriction** of F to R_i (that is, all FDs in F^+ that contain only attributes from R_i) 🙋 or use the original set of dependencies F that hold on R , but with the following test:

- for every set of attributes $\alpha \subseteq R_i$, check that α^+ (the attribute closure of α) either includes no attribute of $R_i - \alpha$, or includes all attributes of R_i .
- If the condition is violated by some $\alpha \rightarrow \beta$ in F , the dependency
$$\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i$$
can be shown to hold on R_i , and R_i violates BCNF.
- We use above dependency to decompose R_i



BCNF and Dependency Preservation

It

is not always possible to get a BCNF decomposition that is dependency preserving

■ $R = (J, K, L)$

$F = \{JK \rightarrow L$

$L \rightarrow K\}$

Two candidate keys = JK and JL

■ R is not in BCNF

■ Any decomposition of R will fail to preserve

$JK \rightarrow L$



Comparison of BCNF and 3NF

- It is always possible to decompose a relation into relations in 3NF and
 - 👉 the decomposition is lossless
 - 👉 the dependencies are preserved
- It is always possible to decompose a relation into relations

in BCNF and

👉 the decomposition is lossless

👉 it may not be possible to preserve dependencies.

5.74



Comparison of BCNF and 3NF (Cont.)

■ Example of problems due to redundancy in 3NF

👉 $R = (J, K, L)$
 $F = \{JK \rightarrow L, L \rightarrow K\}$

J	j_3	$l_1 l_2$
	$null$	K
j_1	L	$k_1 k_1$
j_2	$l_1 l_1$	$k_1 k_2$

A schema that is in 3NF but not in BCNF has the problems of

- repetition of information (e.g., the relationship l_1, k_1) ■
- need to use null values (e.g., to represent the relationship l_2, k_2 where there is no corresponding value for J).

5.75



Design Goals



■ Goal for a relational database design is:

👉 BCNF.

👉 Lossless join.

👉 Dependency preservation.

■ If we cannot achieve this, we accept one of

👉 Lack of dependency preservation

👉 Redundancy due to use of 3NF

■ Interestingly, SQL does not provide a direct way of specifying functional dependencies other than superkeys.

Can specify FDs using assertions, but they are expensive to test

■ Even if we had a dependency preserving decomposition, using SQL we would not be able to efficiently test a functional dependency whose left hand side is not a key.

5.76



Normal Forms



Unnormalized Relation

Only atomic attributes

First Normal Form

Remove nonkey dependency

NF²
1NF
2NF

3NF
BCNF

Second Normal Form

Remove transitive dependency

Third Normal Form

4NF Remove Join Dependencies: 5NF

Dependency preservation: BCNF

Remove Multi-valued Dependencies:

Higher Order Forms



The

Normalization Tree

Pno, Pname, Eno, Ename, Jclass, Chghr, Hrsbilled, Leader

Eno $\square$ Ename, Jclass, Chghr

Eno, Ename, Jclass, Chghr Pno, Pname, Eno*,
Hrsbilled, Leader

Pno $\square$ Pname, Leader Jclass $\square$ Chghr
Leader

Eno, Ename, Jclass*

Pno*, Eno*,
Hrsbilled

Jclass, Chghr

Pno, Pname,



Normalisation Process



Apply a set of normalisation rules to all the attributes of the entity types identified in the data

requirement step.

0NF Relations

1NF Relations

2NF Relations

3NF Relations

**BCNF
Relations**

5.79



Multivalued Dependencies and Fourth Normal

There are database schemas in BCNF that do not seem to be sufficiently normalized



- (a) The EMP relation with two MVDs: $\text{ENAME} \twoheadrightarrow \text{PNAME}$ and $\text{ENAME} \twoheadrightarrow \text{DNAME}$.
- (b) Decomposing the EMP relation into two 4NF relations EMP_PROJECTS and EMP_DEPENDENTS.



Multivalued Dependencies

Consider a database

classes(course, teacher, book)

such that $(c, t, b) \in \text{classes}$ means that t is qualified to teach c , and b is a required textbook for c

- The database is supposed to list for each course the set of teachers any one of which can be the course's instructor, and the set of

books, all of which are required for the course
(no matter who teaches it).